

Computational Verification of BB Code Automorphisms:

A Step-by-Step Correspondence to

*Automorphisms of Bivariate Bicycle Codes via Refined CRT
Components and Local Transfer Data* (outline, AD, HS, JB, IC)

Companion document to `bb.automorphisms` toolkit

Abstract

This document is a line-by-line correspondence between the computational forward-detection and inverse-design algorithms implemented in the `bb.automorphisms` toolkit and the exact theorem, definition, and corollary statements of the outline paper that motivate them. Every paper-derived claim is set in a quotation block, attributed by section or theorem number, and followed immediately by the algorithm step that computationally realizes it. Passages set as *notes* (unboxed) are the author’s own supporting claims used to justify a computational shortcut; they are not verbatim statements from the outline and are flagged as such.

Contents

1	Algebraic preliminaries quoted from the outline	2
1.1	The ambient ring $R_{\ell,m}$	2
1.2	The Fisher–Sehgal trichotomy	2
1.3	The refined CRT decomposition and its “H0” property	2
1.4	Bar involution, circulant lift, and code parameters	3
1.5	The 2-generated stabilizer module	3
1.6	The ring-automorphism catalog	3
2	The computational bridge: row space = cyclic submodule	4
3	Forward problem: automorphism detection	4
3.1	Algorithm F1 — ring-level detection (Corollary 3.6)	4
3.2	Algorithm F2 — full stabilizer-level detection (Theorem 3.2)	5
3.3	Algorithm F3 — catalog construction and scan	6
4	Inverse problem: automorphism design	6
4.1	Algorithm I1 — orbit-closure construction	6
4.2	Algorithm I2 — independent re-verification	7
5	Where shared algebraic structure is used	8
6	Honest scope	8

1 Algebraic preliminaries quoted from the outline

Every algorithm below operates on one of a small number of algebraic objects, all defined directly in the outline. We collect the exact definitions here so that later quotations can refer back to them without repetition.

1.1 The ambient ring $R_{\ell,m}$

“To understand BB codes we must first understand their ambient ring. Where GB codes live in R_ℓ^2 , BB codes instead live in $R_{\ell,m}^2$ where $R_{\ell,m} := R_\ell \otimes R_m$, and

$$R_\ell := \mathbb{F}_2[x]/\langle x^\ell - 1 \rangle$$

(and similarly for R_m , w.r.t the variable y).” Expanding: $R_{\ell,m} = \mathbb{F}_2[x, y]/\langle x^\ell - 1, y^m - 1 \rangle \cong \mathbb{F}_2[x]/\langle x^\ell - 1 \rangle \otimes \mathbb{F}_2[y]/\langle y^m - 1 \rangle = R_\ell \otimes R_m$.”

— OUTLINE, §2.2, eqs. (1)–(2)

“Alternatively, we can view $R_{\ell,m}$ through its group algebra: $R_{\ell,m} \cong \mathbb{F}_2[C_\ell \times C_m]$, where ideals are bivariate abelian codes.”

— OUTLINE, §2.2, eq. (3)

Computational representation. A polynomial in $R_{\ell,m}$ is represented as a dense $\ell \times m$ array over $\mathbb{F}_2$ (class `bb_ring.Ring`), indexed by the monomial basis $\{x^i y^j : 0 \leq i < \ell, 0 \leq j < m\}$ exactly as in eq. (2)–(3) above; ring multiplication is circular (group-algebra) convolution.

1.2 The Fisher–Sehgal trichotomy

“Theorem 2.1 (Fisher–Sehgal [7]). A group algebra $\mathbb{F}_p^k[G]$ is a principal ideal group algebra (PIGA) if and only if the Sylow p -subgroup of G is cyclic.”

— OUTLINE, Thm. 2.1

“With $G = C_\ell \times C_m$, the Sylow 2-subgroup is $C_{2^{s_\ell}} \times C_{2^{s_m}}$ and we arrive at the following trichotomy: [...] in the case that both ℓ, m even, provably at least one ideal is not principal.”

Regime	Sylow 2-subgroup	PIGA?
ℓ, m odd	trivial	yes (and semisimple)
ℓ, m mixed	C_{2^s} cyclic	yes
ℓ, m even	$C_{2^{s_\ell}} \times C_{2^{s_m}}$ non-cyclic	no

— OUTLINE, §2.2

1.3 The refined CRT decomposition and its “H0” property

“Corollary 2.5 ((H0)). $R_{\ell,m} \cong \prod_{c \in C} J_c$ with every J_c finite local, with no dependence on ℓ, m beyond being finite non-negative integers.”

— OUTLINE, Cor. 2.5, “(H0)”

This single fact — that $R_{\ell,m}$, whether or not it is a principal ideal ring (Thm. 2.1), is *always* a finite product of *local* rings — is what licenses every Nakayama-type argument used later, in both the $n = 1$ (Cor. 3.6) and n -generated (Thm. 3.2) cases; see §5.

1.4 Bar involution, circulant lift, and code parameters

“ $\overleftarrow{f}(x, y) := f(x^{-1}, y^{-1})$ ” and “Define the circulant matrix lifting operators: $\rho : x^i y^j \mapsto \text{circ}_\ell(x^i) \otimes \text{circ}_m(y^j)$. Setting $A = \rho(f_1)$, $B = \rho(f_2)$, we obtain the H_X, H_Z parity check matrices: $H_X = [A|B]$, $H_Z = [B^T|A^T]$.”

— OUTLINE, §2.3.1

“Such a code is a $[[2\ell m, k]]$ code, where $k = 2 \dim(R_{\ell, m} / \langle f_1, f_2 \rangle) = 2 \sum_{c \in C} \dim(J_c / \langle f_{1, c}, f_{2, c} \rangle)$.”

— OUTLINE, §2.3.1

`bb_code.BBCode` builds H_X, H_Z exactly via this ρ (`_circulant_lift`), and computes k as $n - \text{rank}(H_X) - \text{rank}(H_Z)$ over $\mathbb{F}_2$ — the direct GF(2)-linear-algebra form of the dimension count above, without evaluating the right-hand component sum $\sum_c \dim(J_c / \langle \cdot \rangle)$ explicitly.

1.5 The 2-generated stabilizer module

“ $\text{StabSpace}(C) = \{(a \cdot f_1, a \cdot f_2, b \cdot \overleftarrow{f_2}, b \cdot \overleftarrow{f_1}) \mid a, b \in R_{\ell, m}\} [\dots]$ This yields an $R_{\ell, m} \oplus R_{\ell, m}$ module, generated by $g_X = (f_1, f_2, 0, 0)$ and $g_Z = (0, 0, \overleftarrow{f_2}, \overleftarrow{f_1})$.”

— OUTLINE, §2.3.1, eqs. (29)–(30)

1.6 The ring-automorphism catalog

“Every group automorphism $\gamma \in \Gamma := \text{Aut}(G)$ extends to an $\mathbb{F}_2$ -algebra automorphism of $R_{\ell, m}$; these are the ring automorphisms realizable as permutations of the $\ell \times m$ qubit grid. The catalog:

1. Multipliers $\psi_{(j_x, j_y)} : x \mapsto x^{j_x}, y \mapsto y^{j_y}, (j_x, j_y) \in (\mathbb{Z}/\ell)^\times \times (\mathbb{Z}/m)^\times$.
2. Partial folds $\theta_x : x \mapsto x^{-1}, y \mapsto y$ and θ_y : involutions folding one axis, with no univariate analogue.
3. The full fold $\iota = \theta_x \theta_y$, i.e. the antipode, the multiplier $(-1, -1)$.
4. Shears: $\text{Hom}(\mathbb{Z}_\ell, \mathbb{Z}_m) \cong \mathbb{Z}_d, d = \gcd(\ell, m)$; for $d > 1$, $x \mapsto xy^{sm/d}, y \mapsto y$ (and transposes) $[\dots]$
5. The transpose $\tau : x \leftrightarrow y$ when $\ell = m$.

”

— OUTLINE, §2.2

“Shifts (multiplication by $x^i y^j$) are deliberately excluded: they are $R_{\ell, m}$ -module automorphisms, not ring automorphisms...”

— OUTLINE, §2.2

`bb_ring.py` implements all five catalog families as explicit maps $(i, j) \mapsto (i', j')$ on exponents (functions `multiplier`, `theta_x`, `theta_y`, `full_fold`, `shear_family` and `shear_family_transpose` for the parenthetical “(and transposes)” above, `transpose_auto`). Item 4’s transpose clause is easy to under-implement — an earlier version of this toolkit omitted `shear_family_transpose` entirely; see §5.

2 The computational bridge: row space = cyclic submodule

This section states, precisely, the one non-paper-quoted fact that the entire computational approach rests on. It is not verbatim in the outline, but it is an immediate consequence of the group-algebra structure the outline itself uses throughout §2.2–§3.

Lemma 2.1 (Rowspace realizes the cyclic submodule). *For $R = \mathbb{F}_2[G]$ a group algebra and $g \in N$ (a free R -module),*

$$R \cdot g = \text{span}_{\mathbb{F}_2}\{s \cdot g : s \in G\}.$$

Proof. Every $r \in R$ is an $\mathbb{F}_2$ -linear combination of group elements $s \in G$ (this is the definition of the group algebra), and the action of R on N is $\mathbb{F}_2$ -linear, so $r \cdot g$ is the corresponding linear combination of the $s \cdot g$. Conversely each $s \cdot g$ is $r \cdot g$ for $r = s \in R$. $\square$

Applied to $g = (f_1, f_2) \in N = R_{\ell,m}^2$: the matrix $H_X = [A|B]$ has one row per group element $s = x^i y^j \in G = \mathbb{Z}_\ell \times \mathbb{Z}_m$ (the row indexed by (i, j) is $s \cdot (f_1, f_2)$, exactly the circulant shift construction of §2.3.1 quoted above), so by Lemma 2.1,

$$\text{rowspace}(H_X) = R_{\ell,m} \cdot (f_1, f_2).$$

That is, H_X 's row space is the cyclic submodule M_X that Thm. 3.1 and Cor. 3.6 (quoted in §3 below) are statements about — not an encoding of it, the object itself, as an $\mathbb{F}_2$ -vector space. This is what lets every module-equality question below be answered by ordinary GF(2) Gaussian elimination.

3 Forward problem: automorphism detection

3.1 Algorithm F1 — ring-level detection (Corollary 3.6)

“(H1) Semilinearity: There exists a ring automorphism $\psi \in \text{Aut}(R_{\ell,m})$ such that, for all $r \in R_{\ell,m}, v \in N$, $\sigma(r \cdot v) = \psi(r) \cdot \sigma(v)$. (H2) Cyclic source: $M = R_{\ell,m} \cdot g$ for a single generator element $g \in N \dots$ ”

— OUTLINE, §3.1

“Assume (H1), (H2). As $R_{\ell,m}$ is local at every component, we have that

$$\sigma(M) = M \iff \sigma(g) = w \cdot g \text{ for } w \in R_{\ell,m}^\times.$$

Proof (Sketch). (H0) means that $\sigma(M) = R \cdot \sigma(g)$ is again cyclic, reducing to a question about when two cyclic modules with given generators coincide. At each component: $M_c = J_c \subseteq N_c$, with g_c the image of g . Apply Nakayama at each c , which we can do as J_c is local even if $R_{\ell,m}$ is not. . . Hence $r_c \in J_c^\times$ at every c , which turns a unit at every component into a global unit.”

— OUTLINE, Thm. 3.1

“Given a BB code C with generator (f_1, f_2) over $R_{\ell,m}$, ϕ is a code automorphism of the code if $\phi((f_1, f_2)) = u(f_1, f_2)$, where u is a unit in $R_{\ell,m}^\times$.”

— OUTLINE, Cor. 3.6

Algorithm F1 decides the right-hand side of Thm. 3.1/Cor. 3.6's “ $\iff$ ” *without* constructing the component decomposition J_c or the unit w itself; it decides the left-hand side directly, via Lemma 2.1, and the theorem's own equivalence licenses treating that as settling the right-hand side too.

Algorithm 1: Ring-level automorphism test (realizes Thm. 3.1 / Cor. 3.6)

Input : code (ℓ, m, A, B) ; candidate $\psi \in \text{catalog}$; swap $\in \{0, 1\}$
Output: TRUE iff ψ (composed with the slot-swap) is a code automorphism

- 1 $A' \leftarrow \psi(A)$; // apply ψ 's exponent-lattice map to every monomial of A
- 2 $B' \leftarrow \psi(B)$;
- 3 **if** swap **then**
- 4 $\lfloor$ swap $A' \leftrightarrow B'$; // tests $\phi((f_1, f_2)) \propto (f_2, f_1)$ instead of (f_1, f_2)
- 5 $H'_X \leftarrow [\rho(A') \mid \rho(B')]$; // same construction as H_X , from §2.3.1
- 6 $r_1, r_2, r_3 \leftarrow \text{rank}_{\mathbb{F}_2}(H_X), \text{rank}_{\mathbb{F}_2}(H'_X), \text{rank}_{\mathbb{F}_2}\begin{pmatrix} H_X \\ H'_X \end{pmatrix}$;
- 7 **return** $r_1 = r_2 = r_3$;
 // $\iff \text{rowspan}(H_X) = \text{rowspan}(H'_X) \iff \psi(f_1, f_2) = w(f_1, f_2), w \in R^\times$ [Thm. 3.1]

3.2 Algorithm F2 — full stabilizer-level detection (Theorem 3.2)

“Assume (H1) holds. $R_{\ell, m}$ is still a per-component local ring. Let M be generated by an n -tuple $g = (g_1, \dots, g_n)$ of elements of N . Then

$$\sigma(M) = M \iff \exists g_{\text{block}} \in M_n(R_{\ell, m}), \sigma(g) = g_{\text{block}} g, \text{ and } g_{\text{block}} \text{ invertible at every } c.$$

”

— OUTLINE, Thm. 3.2

“ g_{ring} elements act as coordinate permutations on a single $R_{\ell, m}$ element. $\dots g_{\text{ring}}$ elements never act on two $R_{\ell, m}$ elements at once. [...] g_{block} elements on the other hand govern interactions between $R_{\ell, m}^b$ module entries and encode transversal, fold transversal, and other low-depth unitary transformations. . . ”

— OUTLINE, §3.1.1

The outline restricts its own worked examples to *scalar* $(0/1)$ g_{block} entries, i.e. $g_{\text{block}} \in M_4(\mathbb{F}_2)$ rather than the fully general $M_4(R_{\ell, m})$:

Swap-fold: $\begin{pmatrix} 0 & I & 0 & 0 \\ I & 0 & 0 & 0 \\ 0 & 0 & 0 & I \\ 0 & 0 & I & 0 \end{pmatrix}$ (“corresponds to swapping on the $(i, i + \ell m)$ fold”). CX-fold: $\begin{pmatrix} I & I & 0 & 0 \\ 0 & I & 0 & 0 \\ 0 & 0 & I & 0 \\ 0 & 0 & I & I \end{pmatrix}$
 (“corresponds to the action of a CNOT controlled on i , targeting $i + \ell m$ ”). Hadamard-type (two variants): $\begin{pmatrix} 0 & 0 & I & 0 \\ 0 & 0 & 0 & I \\ I & 0 & 0 & 0 \\ 0 & I & 0 & 0 \end{pmatrix}$ and $\begin{pmatrix} 0 & 0 & 0 & I \\ 0 & 0 & I & 0 \\ 0 & I & 0 & 0 \\ I & 0 & 0 & 0 \end{pmatrix}$. CZ-fold: $\begin{pmatrix} I & 0 & 0 & 0 \\ 0 & I & 0 & 0 \\ I & 0 & I & 0 \\ 0 & I & 0 & I \end{pmatrix}$ (“CZ’s along the $x^{-1}y^{-1}$ fold”).
 — OUTLINE, §3.1.2–§3.1.4, eqs. (34)–(37)

Algorithm 2: Full stabilizer-module symmetry test (realizes Thm. 3.2, restricted to $\psi_L = \psi_R$ and scalar g_{block})

Input : code; candidate $\psi \in \text{catalog}$; $g_{\text{block}} \in \{\text{eqs. (34)–(37), identity}\}$
Output: TRUE iff $g_{\text{block}} \circ (\psi \oplus \psi \oplus \psi \oplus \psi)$ preserves M

- 1 $S \leftarrow \text{blockdiag}(H_X, H_Z)$; // $2\ell m \times 4\ell m$; **row space** = $M = R \cdot g_X \oplus R \cdot g_Z$ [eq. (29)–(30)]
- 2 **for** $k \leftarrow 1$ **to** 4 **do**
- 3 permute column-block k of S by ψ 's grid map $\rightarrow S_1$; // $\Phi = \psi \oplus \psi \oplus \psi \oplus \psi$, the $\psi_L = \psi_R$ case of §3.1
- 4 **foreach** *output block* $j \in \{1, \dots, 4\}$ **do**
- 5 block j of $S_2 \leftarrow \bigoplus_{i: g_{\text{block}}[j,i]=1} (\text{block } i \text{ of } S_1)$;
- 6 $r_1, r_2, r_3 \leftarrow \text{rank}(S), \text{rank}(S_2), \text{rank}\begin{pmatrix} S \\ S_2 \end{pmatrix}$;
- 7 **return** $r_1 = r_2 = r_3$; // $\iff \exists g_{\text{block}}$ as in Thm. 3.2 with $\sigma(g) = g_{\text{block}}g$

3.3 Algorithm F3 — catalog construction and scan

See the five-item catalog list in §1.6.

— OUTLINE, §2.2 (quoted in full above)

Algorithm 3: Catalog construction (BFS closure) and full scan

Input : (ℓ, m) ; code
Output: all detected symmetries at both levels

- 1 $\mathcal{G} \leftarrow \{\text{all multipliers}\} \cup \{\theta_x, \theta_y, \iota\} \cup \{\text{shear, shear-transpose families}\} \cup \{\tau \text{ if } \ell = m\}$;
 // catalog item 1--5, §2.2
- 2 $\Gamma \leftarrow \text{BFS closure of } \langle \mathcal{G} \rangle \text{ in } \text{Sym}(\mathbb{Z}_\ell \times \mathbb{Z}_m)$, deduplicated by resulting permutation;
- 3 **foreach** $\psi \in \Gamma$, $\text{swap} \in \{0, 1\}$ **do**
- 4 run Algorithm F1(ψ, swap);
- 5 **foreach** $\psi \in \Gamma$, $g_{\text{block}} \in \{\text{eqs. (34)–(37)}\} \cup \{I\}$ **do**
- 6 run Algorithm F2(ψ, g_{block});

4 Inverse problem: automorphism design

“3.4 Automorphism Design” [the outline names this problem but the section itself is currently empty in the draft — Algorithm I1 below is this document’s proposed constructive solution, justified by Cor. 3.6 and Thm. 3.5, both already quoted.]

— OUTLINE, §3.4 (section header only; body not yet drafted in the outline)

4.1 Algorithm I1 — orbit-closure construction

“Theorem: Matching. [...] $\sigma(M) = M \iff \sigma(g) = w \cdot g$ for some $w \in R^\times$.”

— OUTLINE, Thm. 3.5 (restated in §4, “Notation and Theorems to remember”)

“ ϕ is a code automorphism of the code if $\phi((f_1, f_2)) = u(f_1, f_2)$, where u is a unit in $R_{\ell, m}^\times$.”

— OUTLINE, Cor. 3.6 (quoted above)

Cor. 3.6 asks for *some* unit u . Algorithm I1 satisfies it with the simplest possible unit, $u = 1$, by construction: if ϕ literally fixes A as a set of monomials ($\phi(A) = A$), the matching equation holds with $u = 1$ trivially. Building such an A requires no search: take A to be a union of full orbits of the target automorphism group Γ (the group generated by the catalog automorphisms one wants as symmetries) acting on $\mathbb{Z}_\ell \times \mathbb{Z}_m$.

“The antipode permutes C by the involution $c \mapsto c^$, $(\alpha, \beta) \mapsto (\alpha^{-1}, \beta^{-1})$, well defined on Frobenius orbits.”*

— OUTLINE, Prop. 2.7 (“Star pairing”)

Not a direct paper quote. Prop. 2.7 describes how the antipode acts on the CRT component set C ; it does not, in the outline as drafted, state the fact Algorithm I1 actually needs, which is a fact about the *catalog automorphisms themselves*: every catalog map in §2.2 (items 1–5) is $\mathbb{Z}$ -linear on the exponent lattice $\mathbb{Z}_\ell \times \mathbb{Z}_m$, and $\overleftarrow{(\cdot)} = -\text{Id}$ is central in $GL_2(\mathbb{Z})$, so $\phi \circ \overleftarrow{(\cdot)} = \overleftarrow{(\cdot)} \circ \phi$ for every catalog ϕ . This is the author’s own supporting lemma, consistent in spirit with Prop. 2.7 but distinct from it, and it is what licenses setting $B := \overleftarrow{A}$ to get a second (Hadamard-fold) symmetry for free alongside whatever Γ was used to build A .

Algorithm 4: Orbit-closure construction (constructive Cor. 3.6, $u = 1$)

Input : target generators $\varphi_1, \dots, \varphi_k$; seed monomials
Output: code (A, B) exactly invariant under $\Gamma = \langle \varphi_1, \dots, \varphi_k \rangle$

```

1  $\Gamma \leftarrow$  BFS closure of  $\{\varphi_1, \dots, \varphi_k\}$ ; // identical routine to Algorithm F3’s catalog
  closure
2 foreach seed  $(i, j)$  do
3    $\mid$  compute orbit  $\{\gamma(i, j) : \gamma \in \Gamma\}$ ;
4    $A \leftarrow$  sum (in  $\mathbb{F}_2$ ) of the orbit supports;
    // by construction,  $\gamma(A) = A$  for every  $\gamma \in \Gamma$  [Cor. 3.6,  $u = 1$ ]
5   if Hadamard-fold symmetry also wanted then
6      $\mid$   $B \leftarrow \overleftarrow{A}$ ; // free, by the lemma above
7   else
8      $\mid$  repeat orbit construction with a fresh seed for  $B$ ;
9   return  $(A, B)$ 
```

4.2 Algorithm I2 — independent re-verification

Algorithm 5: Re-verification of the constructed code

Input : constructed (A, B) ; group Γ from Algorithm I1
Output: count of Γ -elements confirmed as automorphisms

```

1  $n \leftarrow 0$ ;
2 foreach  $\gamma \in \Gamma$  do
3    $\mid$  if Algorithm F1( $\gamma$ , swap=0) returns TRUE then
4      $\mid$   $n \leftarrow n + 1$ ;
5 return  $n$  (assert  $n = |\Gamma|$ )
```

Algorithm I2 deliberately reuses Algorithm F1 verbatim rather than trusting Algorithm I1’s construction. This closes the loop without being circular: Algorithm F1 has no knowledge of how (A, B) was built, so a pass is independent evidence, not restated assumption.

5 Where shared algebraic structure is used

1. **Corollary 2.5’s “(H0)” licenses both Thm. 3.1 and Thm. 3.2 simultaneously.** Both proofs quoted above invoke Nakayama *componentwise*, because J_c is local even when $R_{\ell,m}$ is not — precisely Cor. 2.5. Algorithms F1 and F2 never construct the components J_c explicitly, but their validity rests on the same single structural fact: it is what makes “row space equal $\Rightarrow$ generators differ by a global unit” true in the first place, for *both* the $n = 1$ and $n = 4$ generator cases.
2. **One routine, `generate_group`, is used by Algorithm F3 and Algorithm I1.** This was not always true: an earlier version of this toolkit gave the forward-scan catalog its own fixed composition template (“one multiplier, one fold, one shear”), which is *not* closed under composition and under-covers the true group generated by the §2.2 catalog items whenever $\ell = m$ (missing interleaved τ -shear compositions) or whenever a shear is present (the “(and transposes)” clause of item 4 was never wired in at all). Unifying the two call sites onto one BFS-closure routine simultaneously fixed the forward scan’s coverage and confirmed the inverse-design code’s group machinery was correct all along.
3. **$\text{rank}(H_X)$ and $S = \text{blockdiag}(H_X, H_Z)$ are computed once per code**, not once per catalog element, and reused as the fixed operand across the whole scan (Algorithm F3’s inner loops).
4. **The circulant blocks $\rho(A), \rho(B), \rho(\overleftarrow{B}), \rho(\overleftarrow{A})$** are computed once (`BBCode._post_init_`) and shared across H_X, H_Z , and S — the same generator data underlies every detection level.
5. **The $\psi_L = \psi_R$ restriction is shared, not just stated.**

“Establish only considering $\psi_L = \psi_R$ in this work...”

— OUTLINE, §3

Algorithm F2 applies *one* permutation to all four column-blocks of S rather than two independent ones, which is both this stated scope restriction and a direct $4\times$ reduction in per-test cost.

6 Honest scope

- The CRT component decomposition itself (J_c , Hensel lifts, local coordinates of §3.2.1) is never constructed; Lemma 2.1 answers the same yes/no question without it, at the cost of never producing the per-component unit ω_c or transport map T_ψ^* the outline’s own proof machinery is built on.
- $\psi_L \neq \psi_R$ (§3.3) is out of scope, matching the outline’s own stated restriction.
- g_{block} is restricted to the six scalar matrices of eqs. (34)–(37); the fully general $g_{\text{block}} \in M_4(R_{\ell,m})$ of Thm. 3.2 is not searched.